编程 agent 的三个层次

第三层

做开发

第 ⑥ 部分

管软件的整个生命周期

第二层

改仓库

第 ④⑤ 部分

一次要动好几个文件

第一层

写代码

第 ②③ 部分

一次补全

agent 的三样原料

- 提示词 — 前几讲讲过
- 工具 — 本讲重点
- 大模型 — 本讲重点

接上一讲

开发者写死的工作流 → Agentless

本讲第 ④ 部分：它为什么一度更好用，又为什么被取代



课件 p.2–3

从提示词写出代码，要会三件事

语言

语法 · API · 惯用写法

```
import json
data = json.loads(s)
for k, v in data.items():
```

编辑

同时看着缺口前后两边

```
total = 0
                    
return total
```

要专门训练

推理

从规格推出程序的行为

```
# 规格：返回最大值
max_of([3, 9, 2])
→ 9  ✓
```

对不对，容易验证

代码例子为示意



课件 p.5–6（中文重绘）

训练分三个阶段

现在没有不会写代码的大模型

编程：语言模型最有价值、用得最广的场景之一

1

预训练

大规模语料
（含代码）

→

2

中期训练

加代码
拉长上下文

→

3

强化学习后训练

推理
以执行结果为奖励

这一段讲 ① ② 两个阶段；③ 放到评测那一段再讲



课件 p.5–6（中文重绘）

预训练：混数据，保结构

- 网页文本
- 技术文字
- 数学 练逻辑推理
- 源代码
- 文档

→ 混合语料

保住结构

缩进 文件边界 文字和代码的关系

原文（示意）

```
def area(r):
····if r > 0:
········return r * r
```

早年处理：一串空格压成一个

```
def area(r):
·if r > 0:
·return r * r
```

× 缩进层级没了

$$L_{\text{mix}} = \mathbb{E}_{x \sim D_{\text{mix}}} \left[-\Sigma_t \log p_\theta(x_t \mid x_{<t}) \right]$$

→ 训练目标：预测下一个 token

边训边按领域分开算损失 + 真正要跑的编程任务



课件 p.7–8（中文重绘）

洗代码数据，比洗文本还难

1 过滤

按语言和来源：去掉生成的转储、编码损坏、低价值重复

2 去重

fork　第三方库　几乎一样的文件　抄来的答案

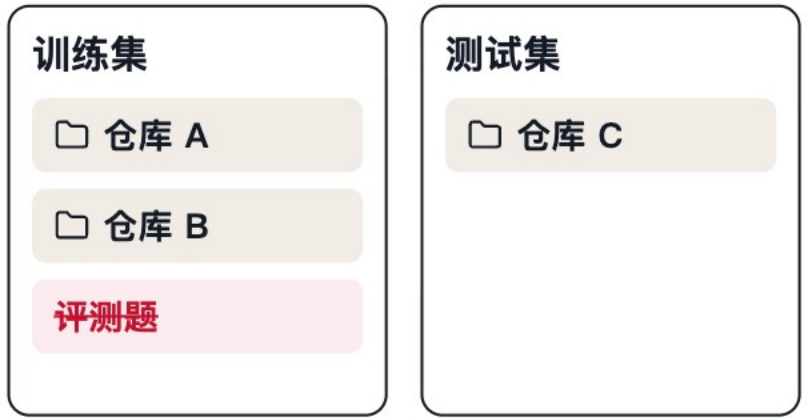
3 管住来源

许可证　源码日期　退出请求　敏感数据

4 防泄漏

按仓库切分　清掉评测题

按仓库切分，不按文件



评测题从训练数据里清掉（示意）



课件 p.7–8（中文重绘）

把代码切成 token

字节级 BPE · 代码加文字

空白要保留

常见的一串空白，合成一个 token

StarCoder2 分词器 · 实际切法

源码

```
if·ready:↵····return·value↵
```

token

if	·ready	:	↵···	·return	·value	↵

· 是空格，↵ 是换行，每个方块是一个 token

以前：一个空格一个 token（示意）

↵	·	·	·	·	·	·	·	·	return



课件 p.9（中文重绘）

空白能不能跨着合并?

要不要给空白专门加 token?

✓ BPE 本来就在合并字节，空白不用特殊对待

→ 可几乎所有人还是 **把空白和非空白分开切** ： 免得吐出半个词

如果允许跨空白合并

`import·numpy·as·np`

更紧凑：InCoder 试过，token 数少约四成

用户只打了半句（示意）

```
import numpy as
```

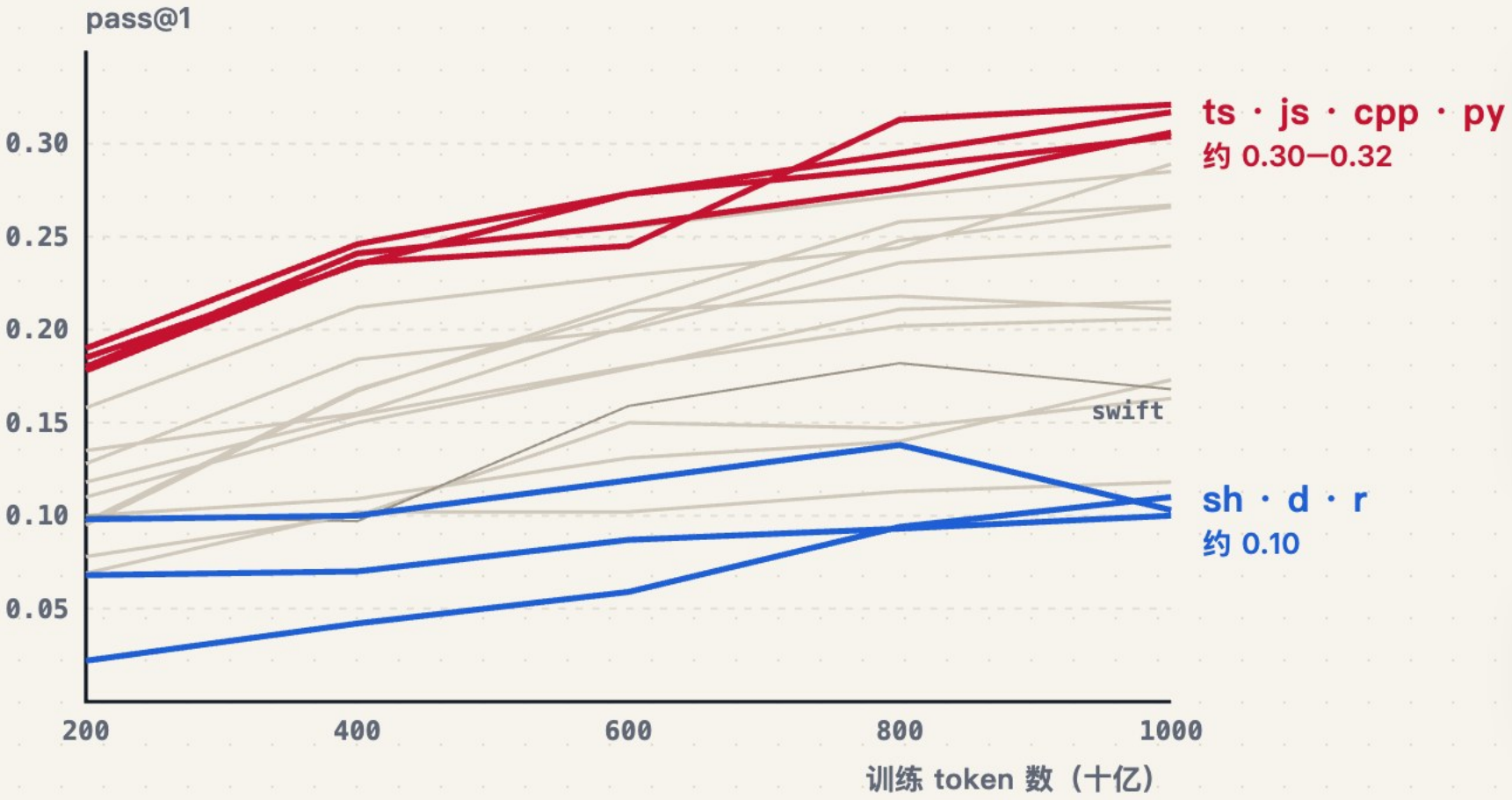
?

补不出 → 退回去做受约束的解码



课件 p.9（中文重绘）

大多数语言，训得越多越好



StarCoderBase

155 亿 参数

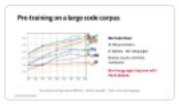
1 万亿 token

80 多种语言

数据：源码、issue、提交记录、notebook

曲线按原图读数

训练数据多的语言，自然做得更好



课件 p.10（中文重绘）

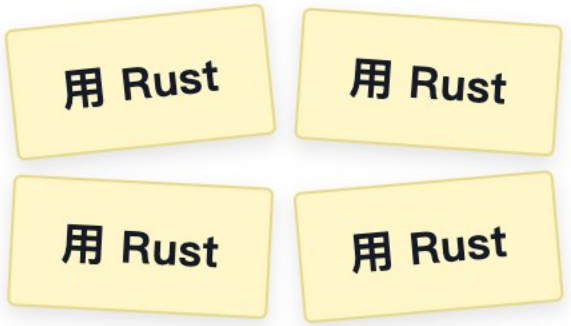
模型偏爱 Python

一条很重的数据处理流水线（示意）

~~Python~~ → Rust 改写完成，提速

之后每次跑程序

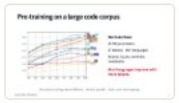
```
$ python pipeline.py
```



简洁好写

流行

强化学习多半配着 Python 解释器做



课件 p.10 · 课上补充

中期训练：加大代码比例

继续做语言模型训练

配比为示意，不按真实比例

预训练



中期训练



代码比例 ↑　文本和数学能力要保住

Code Llama

课件 p.11



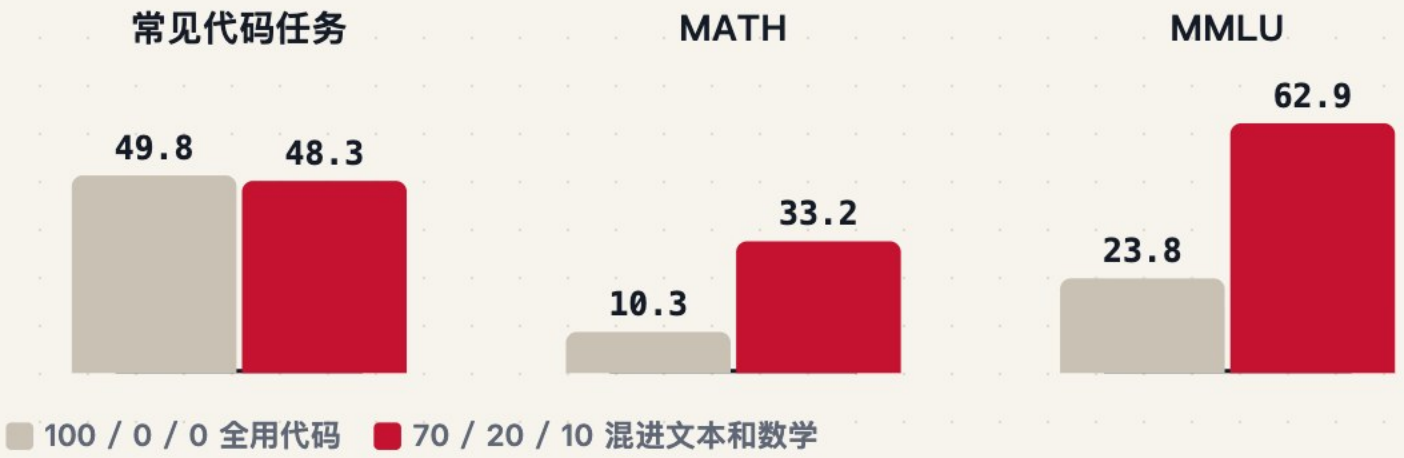


课件 p.11–12（中文重绘）

代码、文本、数学怎么配

代码 / 文本 / 数学	常见代码任务	MATH	MMLU
100 / 0 / 0	49.8	10.3	23.8
70 / 20 / 10	48.3	**33.2**	**62.9**

Qwen2.5-Coder 表 3 节选 · 各列最高值加粗



现在没人只拿一个模型写代码：文本和数学能力要保住



课件 p.11–12（中文重绘）

只看一个文件，看不出来

types.py

Result.value
可能是 None

```
value: int | None
```

→

worker.py

把 Result
往下传

```
return run(job)
```

→

caller.py

要补一个
空值检查

```
if r.value is None:
```

只看一个文件，根本看不出来

代码为示意

Qwen2.5-Coder 的上下文长度



- ✓ 验证：跨文件补全
- ✓ 验证：填空
- ✓ 验证：短上下文能力没丢



课件 p.13（中文重绘）

填空：同时用缺口前后的代码

```
def is_positive(x: int) -> bool:
    return x > 0
```

课件例子 · 示意

✎ 编辑能力从哪来？ 填空训练 ：同时用缺口前后的代码

InCoder：最早一批用填空目标训练的强代码模型



课件 p.14–15（中文重绘）

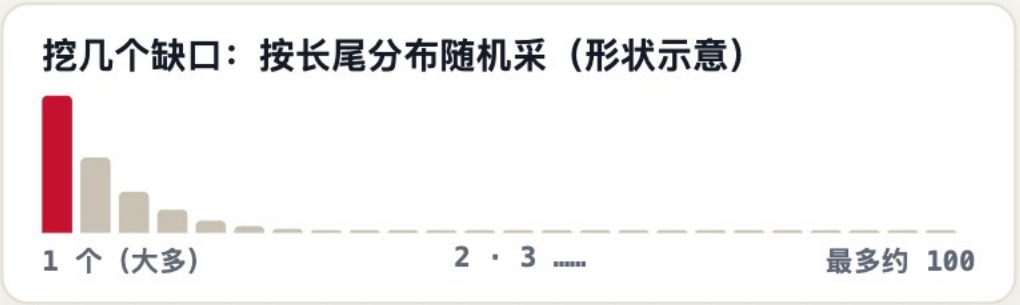
做法：挪到最后，放个哨兵

原文 前缀 | 片段 | 后缀

训练 前缀 [M0] 后缀 [M0] 片段 [END] ← 片段挪到最后

推理 前缀 [M0] 后缀 [M0] → 生成片段

多个缺口 前缀 [M0] 中段 [M1] 后缀 → [M0] 片段 0 [M1] 片段 1 [END]



都是填空的特例

- 在大文件中间补代码
- 推断返回类型
- 写注释
- 重命名变量



课件 p.14–15（中文重绘）

右边的上下文，值多少

由 HumanEval 改成的填空题 · 补全的函数通过测试的比例

推理方法	后缀用来	单行	多行
从左往右 · 生成 1 个	不用	48.2%	24.9%
从左往右 · 生成 10 个	只用来挑	54.9%	28.2%
因果掩码填空	用来生成	**69.0%**	**38.6%**

训练目标 · 同样 520 亿 token	HumanEval	MBPP
从左往右的语言模型	6.0%	8.9%
因果掩码填空	8.0%	10.9%

而且完全不用挑

不降反升

普通的从左往右补全没被耽误（pass@1）



课件 p.16–17（中文重绘）

填空之外，代码里还藏着别的信号

提交的 diff OctoPack

改前代码 + 提交说明 → 改后代码

诊断信息 DrRepair

坏代码 + 报错 → 修复

测试 CodeRL

程序 + 测试结果 → 奖励

执行轨迹 CodeExecutor

程序 → 执行过的行和状态

给程序插桩（示意）

```
x = 3
if x > 0:          x = 3
    y = x * 2      y = 6
else:
    y = 0
print(y)           输出?
```

预测执行到某一步的输出



课件 p.18（中文重绘）

和参考答案比

精确匹配

生成的代码和参考答案是不是一字不差

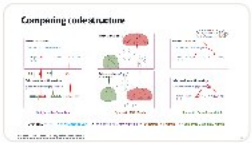
BLEU-4

算 token 的 n 元组重叠，太短还要罚分

判断整数 x 是不是正数

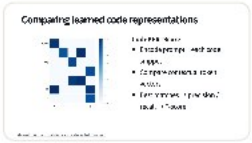
参考答案	`return x > 0`	规格
字面只改一点	`return x >= 0`	✕ x = 0 时错
写法不同	`return not (x <= 0)`	✓ 对整数等价

后来的改进



CodeBLEU

n 元组 + 语法树 + 数据流



CodeBERTScore

token 向量逐个对上



课件 p.20–22（中文重绘）

跑起来，看行为

候选代码

```
def is_positive(x):
    return x >= 0
```

测试用例

输入	-1	0	1
期望	F	F	T

→

隔离环境

时间、内存限制

→

输入	实际	期望	
-1	F	F	✓
0	T	F	×
1	T	T	✓

失败：0 不是正数

全部通过记 1 分，有一个不过就是 0 分

各种正确的写法都接受

执行时抓到语义错误



课件 p.23–24（中文重绘）

执行检查的局限

三条局限

1. 有限的测试会漏掉错误行为
2. 要可信的测试、依赖和隔离执行
3. **要时间和稳定的执行环境**

训练编程 agent 最大的难题之一

任务越来越难，测试要同时压低假阳性和假阴性

依赖装在 Docker 里（示意）



大仓库跑一遍测试，很费时间



课件 p.24（中文重绘）

两个经典基准

HumanEval

已经过时

- 164 个 Python 函数
- 文档字符串 → 补全
- 隐藏的单元测试

CodeContests

- 竞赛题，含 Codeforces
- 题面 → 完整程序
- 编译 + 测试

据说

是 OpenAI 的人手写的一批有点刁钻的题；现在每个模型都能拿高分

同一道题，一次次提交

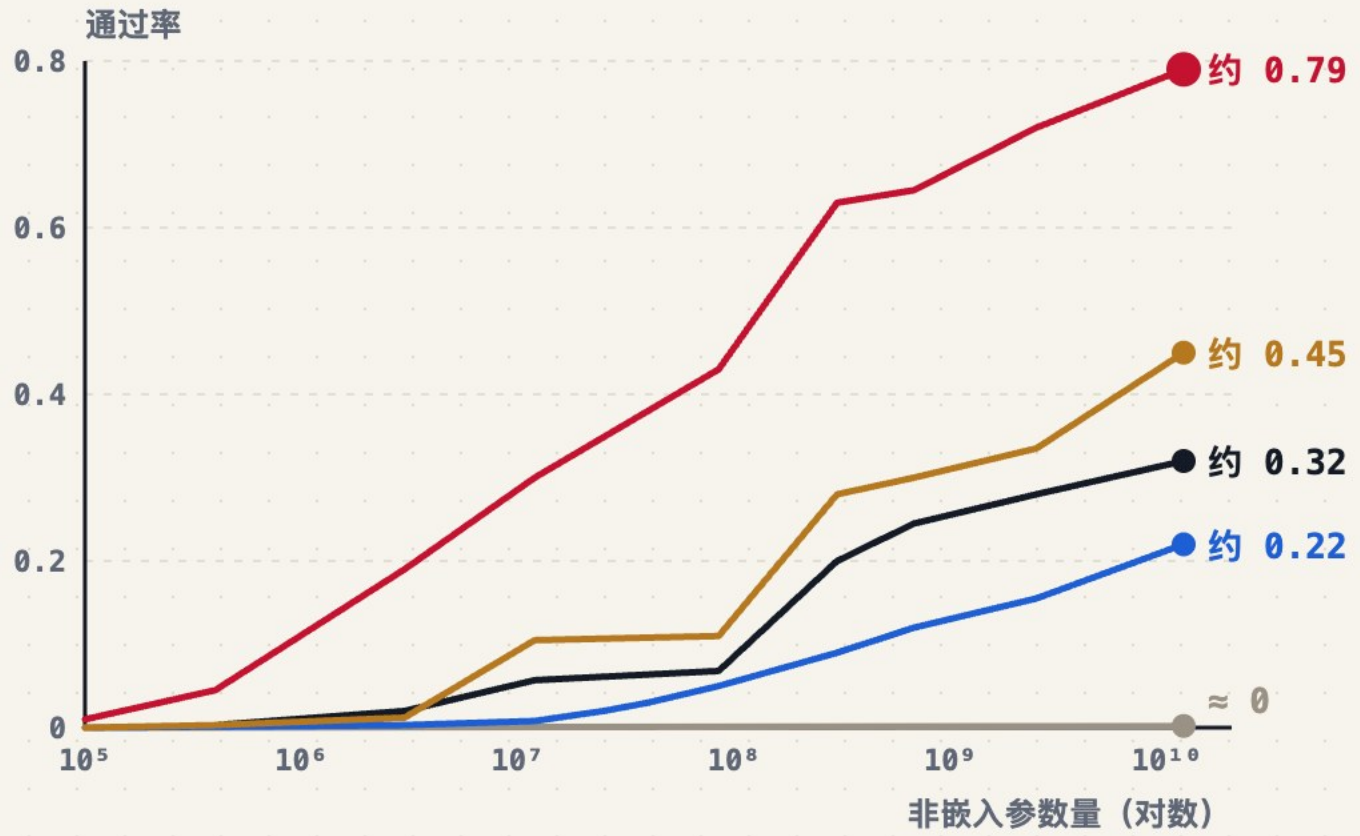


第一次就全对，很难



课件 p.25（中文重绘）

能从多个里挑，比只生成一个高得多



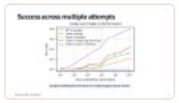
pass@1 只生成一个，就要通过

pass@k 采样 k 个，至少一个通过全部测试

这里的 Codex = 2021 年给 GitHub 做的补全模型

- GPT-3 pass@1
- Codex pass@1
- Codex-S pass@1
- 按平均对数概率重排
- 按神谕重排

最大的模型上 · 按原图读数



课件 p.26（按原图读数）

对不对之外：快不快、好不好维护

NoFunEval · 397 个编辑任务

现有程序

测试都能过

耗时 [redacted]

→

代码模型

"让这段代码更快"

→

改后的程序

输出不变

耗时 [redacted]

要求的改进	怎么查	分数
运行效率	跑测试，计时	加速比
延迟和资源占用	和参考修改比	DiffBLEU
可维护性和安全	静态检查 + 修改匹配	CodeQL × DiffBLEU

非 agent 任务：训练时基本只看对不对

agent 任务：考虑效率更常见



课件 p.27（中文重绘）

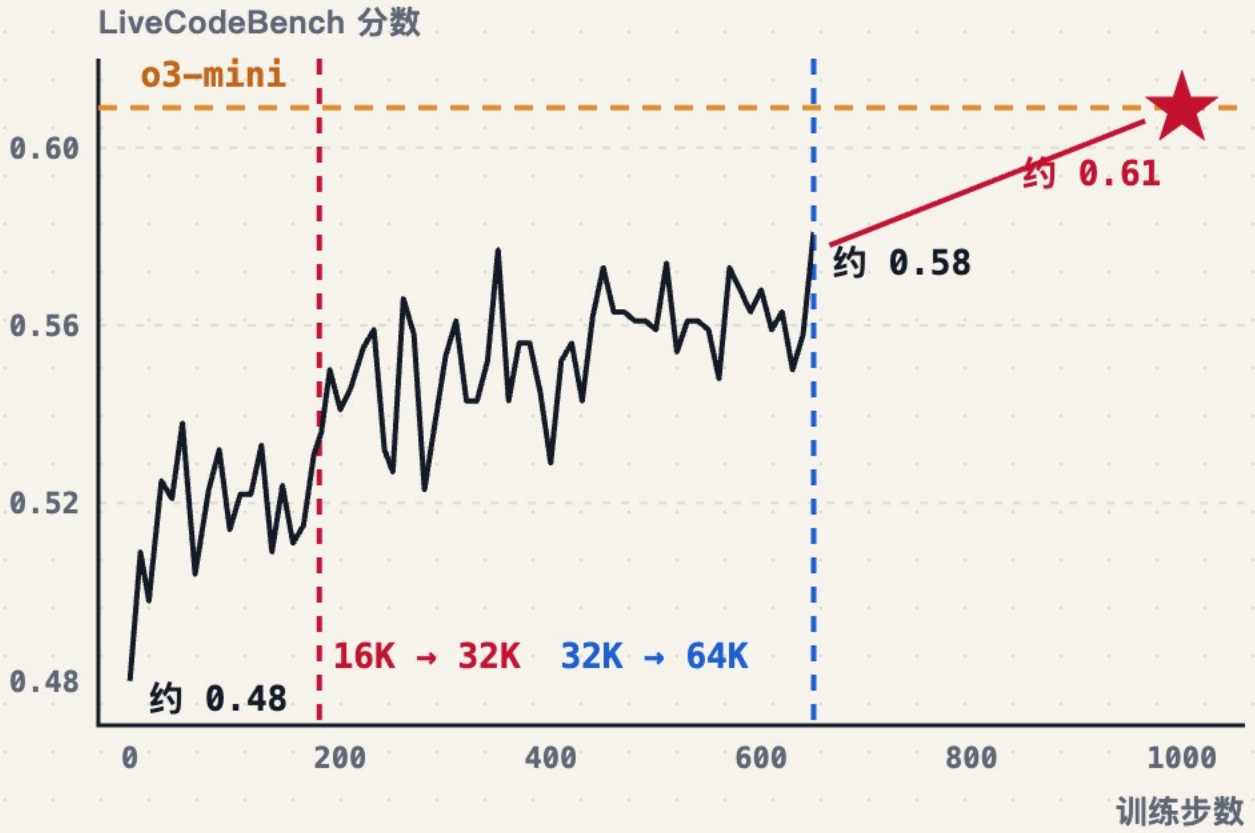
DeepCoder：上限一路提高，分数一路涨

采样 → 跑测试 → 奖励 → 更新 ↺

- 训练时回答的 token 上限：16K → 32K
- 再提到 64K
- ★ DeepCoder-14B：评测时给 64K，不再训练
- o3-mini（参考虚线）

曲线和数值都按原图读数

0.48 → 0.58 → 0.61





课件 p.28、p.30（按原图读数）

模型为什么爱写注释?

只能猜

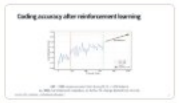
生成的代码（示意）

```
def mean(xs):          # 定义函数
    total = 0          # 初始化总和
    for x in xs:       # 遍历列表
        total += x     # 累加
    n = len(xs)        # 取长度
    return total / n   # 返回平均值
```

1. **现有代码就有注释**
   模型在现有代码上训练
2. **边写边想**
   多花些 token，想清楚下一步
3. **代码质量模型要求数量合适**
   据说较新的 Codex 发布时专门强调过

可读性评分器 → 奖励 → 后训练

或：人对可读性的评判 → RLHF 奖励模型



课件 p.30 · 课堂问答

从单步编程，到 agent 式编程

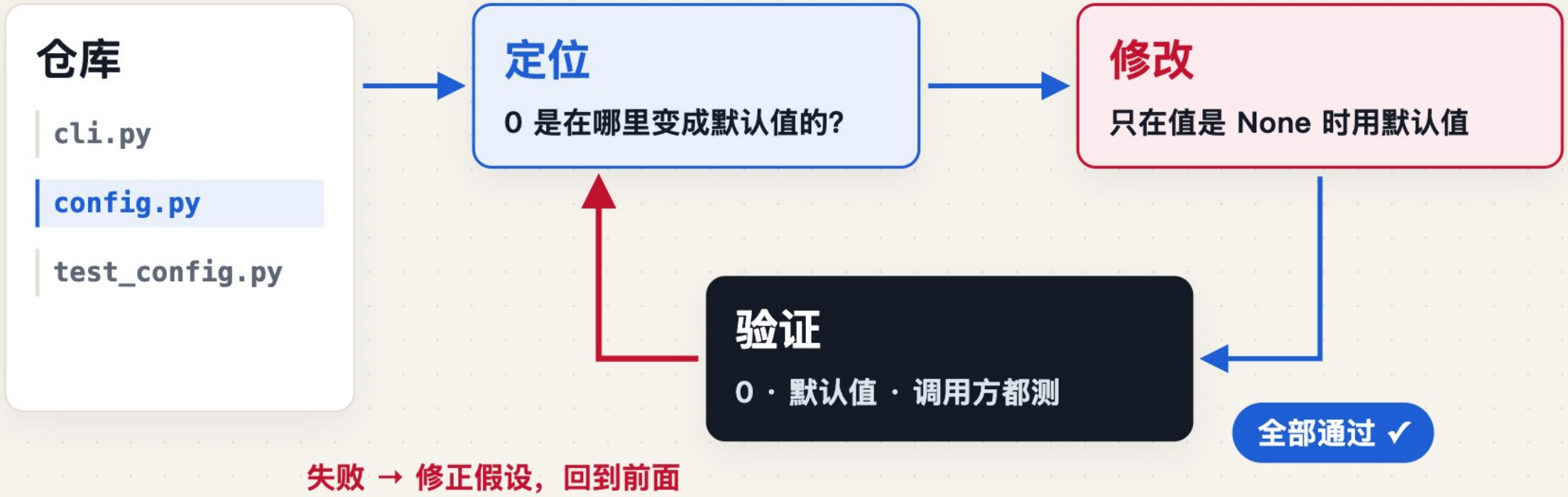
单步编程	自成一体的提示词	推理 + 代码	评测答案
agent 式编程	issue + 已有文件	搜索 → 修改 → 跑测试	补丁 + 证据

最常见的场景：在代码仓库里修 bug、做新功能



课件 p.32–33（中文重绘）

定位 → 修改 → 验证，一个循环



教学示意（据 SWE-agent 交互循环）



课件 p.32–33（中文重绘）

zsh — retry-demo

```
$ rg 'retries' buggy_config.py
return config.get("retries") or 3
$ RETRY_MODULE=buggy_config python3 \
  -m unittest test_config.RetryTests.test_zero
AssertionError: 3 != 0
FAILED (failures=1)
```

去修条件，不改测试



课件 p.34–36（示意）

```
es")
 else value
```

```
$ rg 'retries' buggy_config.py
return config.get("retries") or 3
$ RETRY_MODULE=buggy_config python3 \
  -m unittest test_config.RetryTests.test_zero
AssertionError: 3 != 0
FAILED (failures=1)
$ python3 -m unittest -v test_config
test_missing ... ok  # 缺省 → 3
test_none ... ok  # None → 3
test_zero ... ok  # 0 → 0
test_positive ... ok  # 正数 → 2
Ran 4 tests
OK
```



课件 p.34–36（示意）

另一种做法：固定工作流 Agentless

现在：基本大家都用 agent

issue：lambdify 把某些矩阵表达式解释错了

① 代码库

定位

② 仓库结构
挑出前 N 个文件

③ 嵌入检索
补充文件

④ 类和函数
class · def

⑤ 要改的位置
具体那几行

修复

⑥ 多个候选补丁
单步模型生成

补丁验证

⑦ 生成复现测试

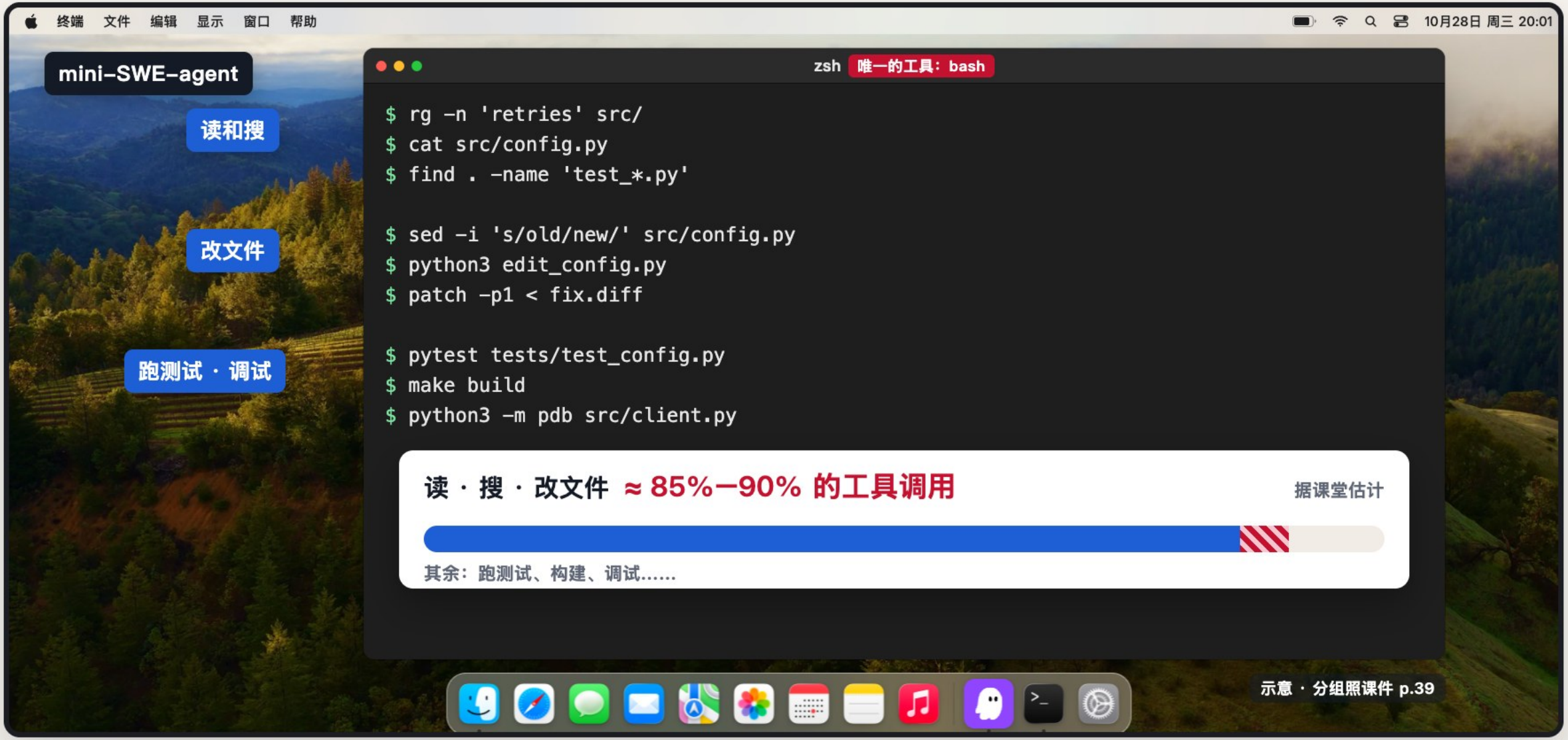
⑧ 跑测试

⑨ 过滤 · 排序

⑩ 提交
一个补丁



课件 p.37（中文重绘）


终端 文件 编辑 显示 窗口 帮助
10月28日 周三 20:01
mini-SWE-agent
读和搜
改文件
跑测试 · 调试
zsh
唯一的工具：bash
$ rg -n 'retries' src/
$ cat src/config.py
$ find . -name 'test_*.py'
$ sed -i 's/old/new/' src/config.py
$ python3 edit_config.py
$ patch -p1 < fix.diff
$ pytest tests/test_config.py
$ make build
$ python3 -m pdb src/client.py
读 · 搜 · 改文件 ≈ 85%–90% 的工具调用
据课堂估计
其余：跑测试、构建、调试……
示意 · 分组照课件 p.39



课件 p.39–40（示意）

```
$ find . -name 'test_*.py'

$ sed -i 's/old/new/' src/config.py
$ python3 edit_config.py
$ patch -p1 < fix.diff

$ pytest tests/test_config.py
$ make build
$ python3 -m pdb src/client.py

$ sed 's/^RETRIES = 3$/RETRIES = 5/' \
    settings.py > settings.new
$ mv settings.new settings.py
$ sed -i 's/\\\\\\\\n/\\\\n/g' fmt.py  # 示意
```

个数错一个就不对

bash 不一定是改文件最好的接口

示意 · 分组照课件 p.39



课件 p.39–40（示意）

改文件的三种格式

bash 之外，再加一个改文件的工具

① 整文件重写

```
settings.py
RETRIES = 5
TIMEOUT = 30    没改，也要再抄
```

用起来简单；没改的代码要再抄一遍

Pi · OpenCode 的 write · OpenHands 的 create

```
文件一长（示意）
import os
HOST = "localhost"
PORT = 8080
RETRIES = 5    ← 只改这行
TIMEOUT = 30
LOG_LEVEL = "info"
……
```

改一行，也要把整个文件发一遍

② 查找替换

```
<<<<<<< SEARCH
RETRIES = 3
=======
RETRIES = 5
>>>>>>> REPLACE
```

紧凑；但旧的那段要能唯一匹配

OpenHands · Pi · OpenCode 的 edit

```
settings.py（两处一样，示意）
# 生产
RETRIES = 3
# 测试
RETRIES = 3                    ✓ 唯一
```

✓ 几乎所有编程 agent 都用它

③ 统一 diff

```
--- a/settings.py
+++ b/settings.py
@@ -1,2 +1,2 @@
-RETRIES = 3
+RETRIES = 5
 TIMEOUT = 30
```

打不上

要写行号：agent 写的行号总是错

Codex 补丁语法

不写行号

```
*** Begin Patch
*** Update File: settings.py
@@
-RETRIES = 3
+RETRIES = 5
 TIMEOUT = 30
*** End Patch
```

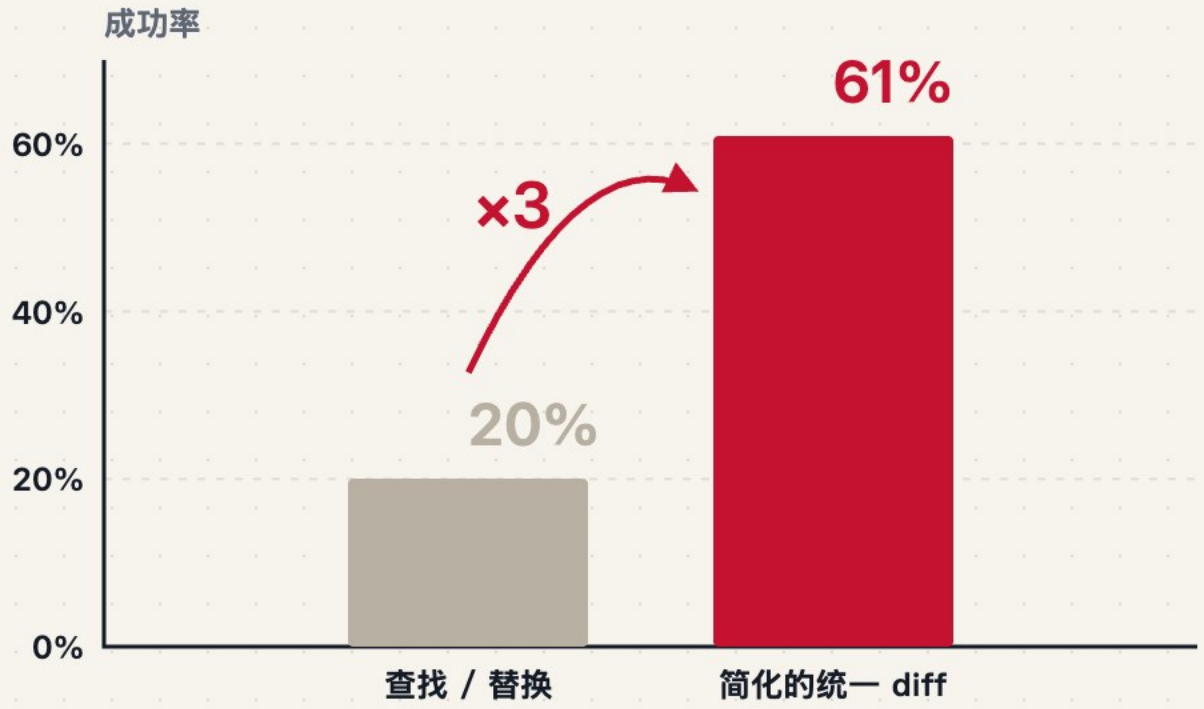
Aider：省掉每段行号的 diff



课件 p.41–42（中文重绘）

格式的差别，可能很大

Aider 重构基准 · 89 个 Python 任务 · GPT-4 Turbo（gpt-4-1106-preview）



数据：课件 p.43（Aider 的统一 diff 实验）

每个任务要做的事

1. 大类里的一个方法
2. 挪成顶层函数
3. 检查语法、代码有没有保留

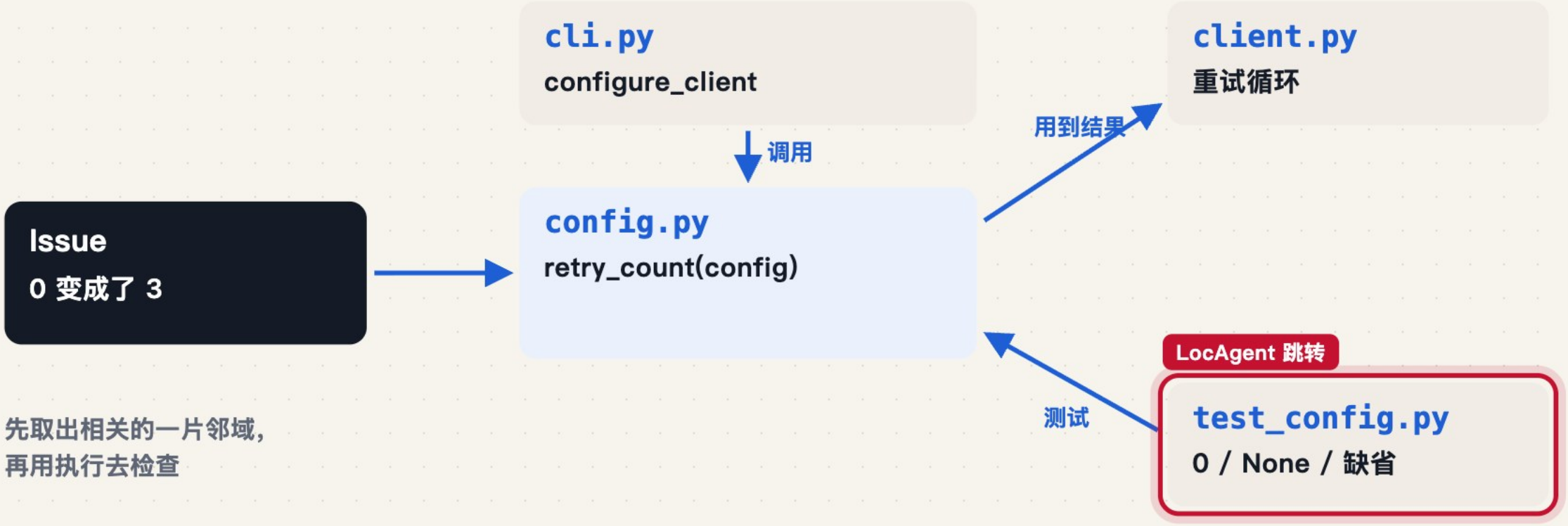
现在差别没那么大，但依然存在

据说：Gemini 用自己那套编辑格式训练，在不少脚手架上表现很差，大概几个月前才修好



课件 p.43（中文重绘）

顺着依赖图走：LocAgent



先取出相关的一片邻域，
再用执行去检查



课件 p.44–45（中文重绘）

SWE-bench 怎么评

代码库
`astropy/astropy`

issue
对结构化数组表做 `vstack` 时，报类型转换错误

测试
`table/tests/test_ops.py`

语言模型

生成补丁
`astropy/utils/meta.py`
`- return arr.dtype.str`
`+ return arr.dtype.str if …`

测试框架
打补丁 → 打测试
→ 跑测试脚本
→ 记结果

结果
`test_join_struct_col`
`test_vstack_struct_col`
`test_dstack_struct_col`

失败 → 通过：要求的修复

通过 → 通过：原有行为保住

全过记 1 分，否则 0 分

Django、NumPy：整套测试跑一遍要 5–10 分钟甚至更久
→ 只挑一小组相关的测试

harness = 只跑测试
不是 agent 的脚手架



课件 p.47（中文重绘）

做成能跑的训练任务

评测能自动跑 = 能拿来训练

一个能跑的训练任务，要三样

起始状态	文件 + 依赖 + 测试命令
issue	要求的行为 + 怎么复现
检查	修之前失败，修好后恢复

先验证环境真能用，再收集训练轨迹

搜索 ? → 找到相关文件

→ 修改 ? → 改出补丁

→ 测试 / 结束 ★ 奖励 → 奖励只在最后给：修好没有

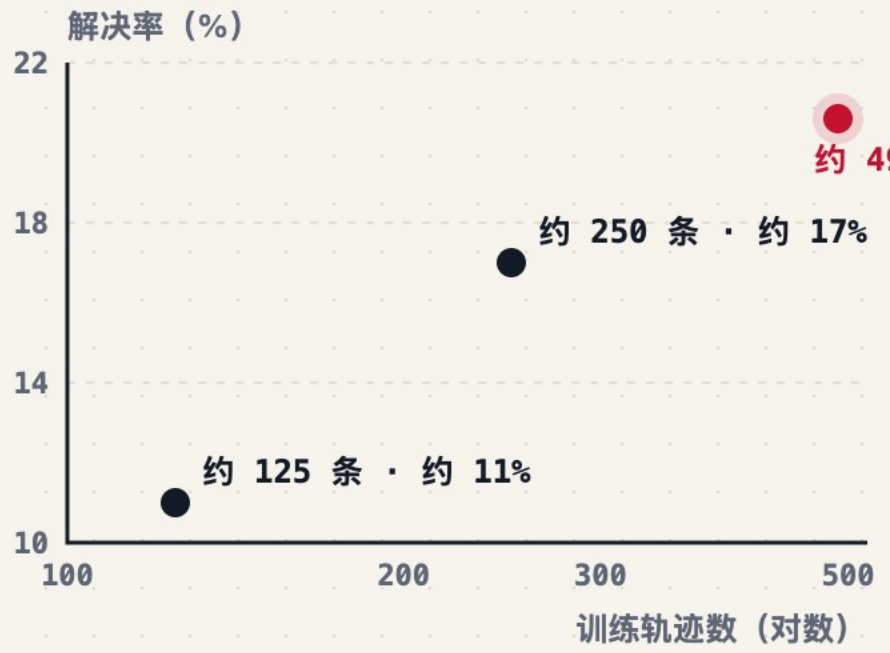


课件 p.48–49（中文重绘）

SWE-Gym：训练数据和推理算力

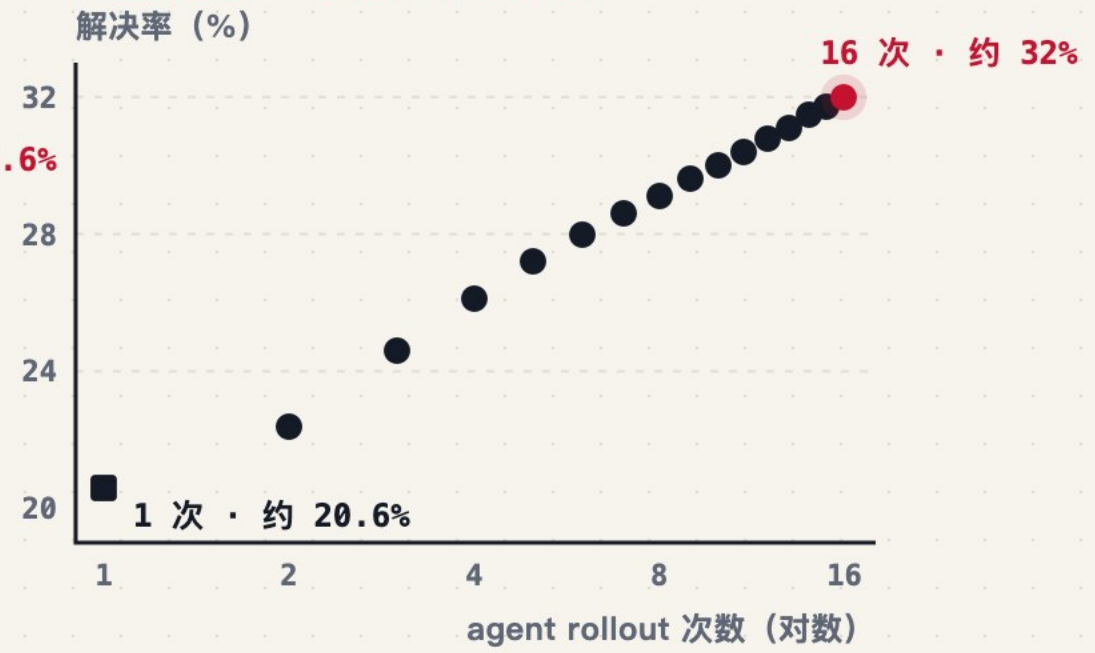
SWE-bench：只有评测集配好了环境 → SWE-Gym：另外做了训练用的　　32B 模型 · SWE-Bench Verified

训练规模

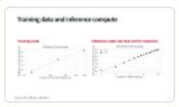


按原图读数

推理规模：用学到的验证器挑



按原图读数



课件 p.50（按原图读数）

SWE-smith：自己往代码里注入 bug

1
真实仓库
`src/` · `tests/`
单元测试全部通过

→

2
建环境
SWE-agent 装仓库、跑测试
→ Dockerfile

→

3
生成策略
程序化修改 · 模型生成
组合 bug · PR 镜像

→

4
新任务实例
环境镜像 + 生成的 issue
带 bug 的补丁 + 验证过的测试

准备能执行的基线 → 改代码，不改测试 → 留下新出现的测试失败

借来的想法：变异测试

故意改坏代码 → 测试还能过?
→ 说明测试有漏洞

+ 非常轻量，几乎能造无限多的数据

– 不一定是自然的 bug

注入一个 bug（课件 p.52）

```
value = config.get("retries")
- return 3 if value is None else value
+ return 3 if not value else value
```

改之前 4 过 → 变异后 3 过 · 1 挂 → 修好后 4 过

请修好它



课件 p.51–52（中文重绘）

跨语言：先读项目配置

语言	项目配置	测试命令
<> Python	`pyproject.toml`	`pytest`
<> TypeScript	`package.json`	`npm test`
<> Rust	`Cargo.toml`	`cargo test`

好在现在的编程 agent 自己就很会处理这些约定



课件 p.53–54（中文重绘）

多套脚手架，一起训

脚手架 = 提示词 + 工具 + agent 循环 + 上下文管理

模型期待的编辑格式和脚手架对不上，表现就会变差

共享模型
同一份权重

- **OpenHands** 补丁 → 测试奖励
- **OpenCode** 补丁 → 测试奖励
- **Codex** 补丁 → 测试奖励

一起更新模型
轨迹 + 奖励

Nemotron 3
课件 每类任务至少 2 套
课上 大概 5 套以上



课件 p.53–54（中文重绘）

前端也是定位、修改、验证的循环

定位

找到组件和
事件处理函数

修改

改逻辑和样式

在浏览器里验证

操作页面

输入 → 点击 → 查保存下来的状态

看渲染出来的样子

截图 → 视觉理解

检查失败，就回去修改

SWE-bench Multimodal：SWE-agent + 浏览器 + 截图工具

只修前端 bug，还是修 issue



课件 p.56

在浏览器里验证，两条路

浏览器 agent

每一步自己选下一个动作

观察 → 选动作

↓ ↑

在浏览器里输入 / 点击

自适应地探索 · 下一讲专门讲

Playwright 脚本

模型写一个可重复的检查

1	名字框里填空格	`fill()`
2	点"保存联系人"	`click()`
3	断言保存的状态	`expect()`
4	截图	`screenshot()`



课件 p.57

界面为示意 · 照课件 p.58 重绘

ew

ContactForm.js — contacts-app

js ContactForm.js

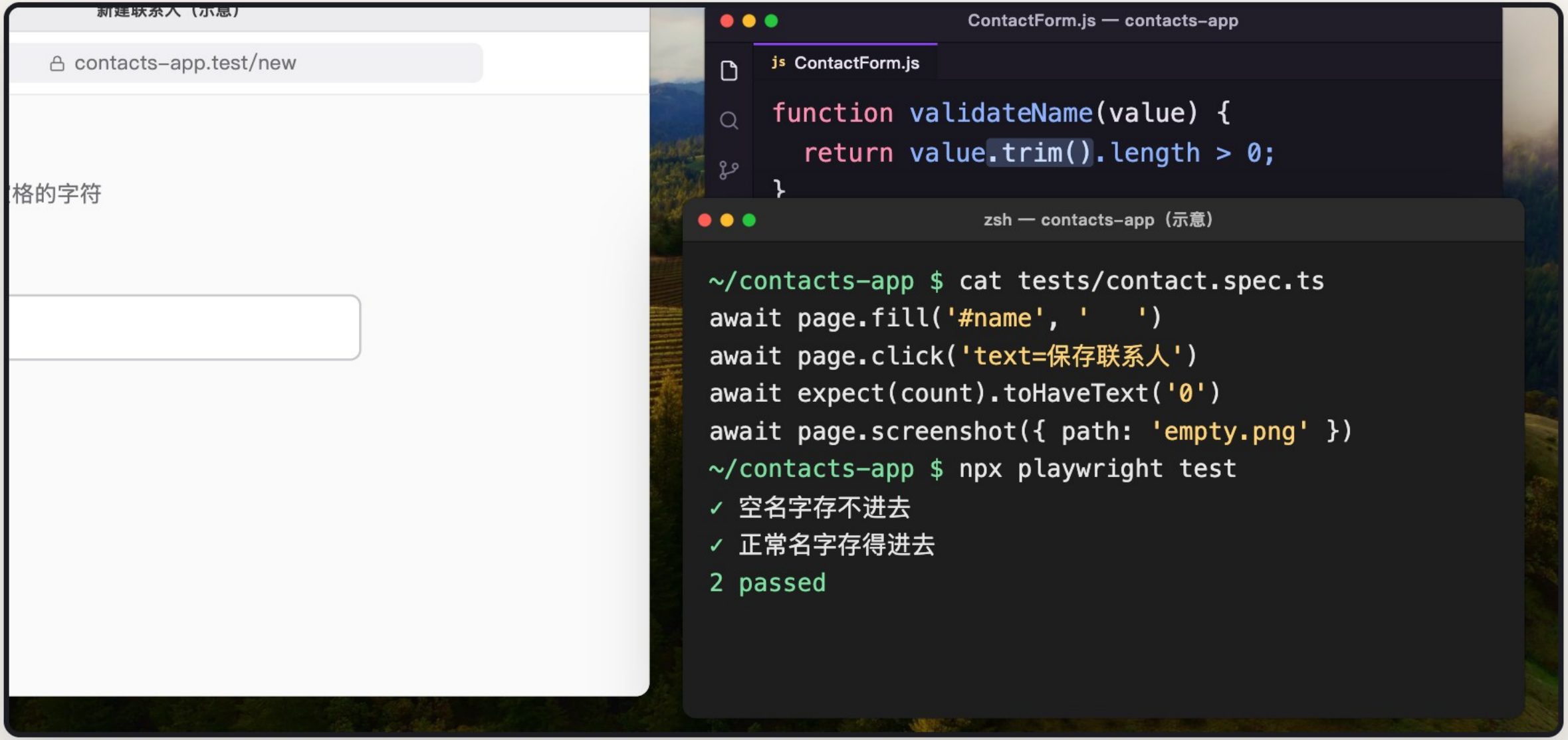
```
function validateName(value) {
  return value.length > 0;
  return value.trim().length > 0;
}

function onSave(form) {
  if (!validateName(form.name)) {
    showError('name', '请输入非空的名字');
    return;
  }
  saveContact(form);
}
```

只改一处：先去掉两头的空格



课件 p.58（界面示意）


新建联系人（示意）
contacts-app.test/new
格的字符
ContactForm.js — contacts-app
ContactForm.js
function validateName(value) {
return value.trim().length > 0;
}
zsh — contacts-app（示意）
~/contacts-app $ cat tests/contact.spec.ts
await page.fill('#name', ' ')
await page.click('text=保存联系人')
await expect(count).toHaveText('0')
await page.screenshot({ path: 'empty.png' })
~/contacts-app $ npx playwright test
✓ 空名字存不进去
✓ 正常名字存得进去
2 passed



课件 p.58（界面示意）

带图的 issue，还是跑检查

~~按图片相似度打分~~

issue + 截图

观察到：空名字被保存
期望：拒绝它

→ **起始仓库**

→ **候选补丁**
改校验

→ **执行检查**
空名字 → 报错
没有新增记录

回归检查
合法名字 → 能保存
已有测试通过

⚠ issue 里有图片，不等于就用图片相似度打分



课件 p.59（中文重绘）

GLM 5.2：不是多模态模型，拿了第一

前端设计竞技场 · 排行榜（示意，不标分数）

1	GLM 5.2	不是多模态模型
2		
3		
4		

"做好前端，
一定要多模态"？

被动摇了

不过

多模态能力多半还是挺重要的



课件 p.59（中文重绘）

测试通过只是一步，不是发布决定

外循环：其余的一切

需求
用户需要什么
兼容性

内循环

实现循环
修改 ⇄ 测试

评审
diff + 证据

发布决定

部署 / 监控
生产信号
回滚路径

维护
新需求
观察到的故障

测试通过只是其中一步，不是发布决定

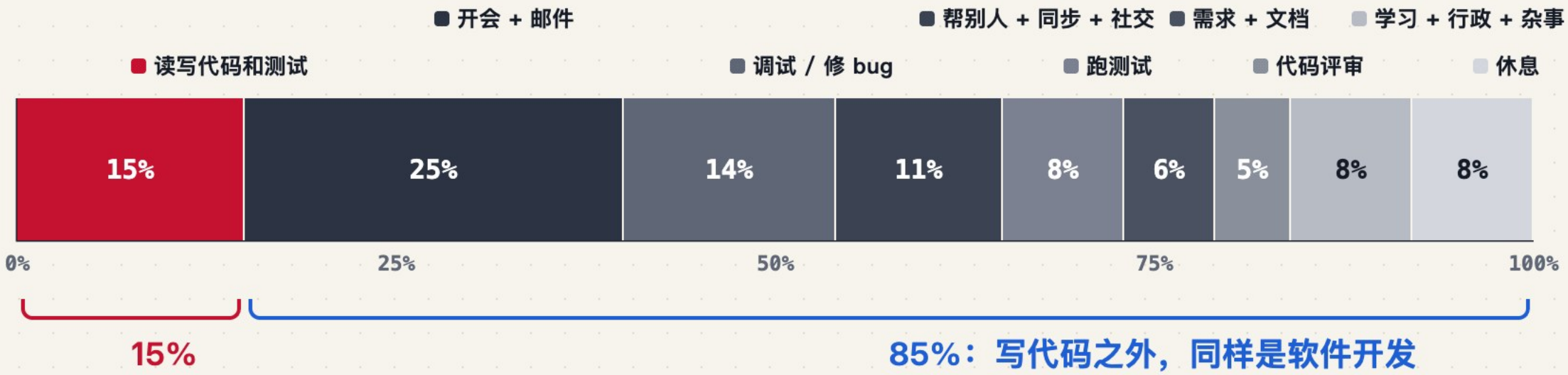
理想情况：agent 都能帮上 更难定义 更难找训练数据



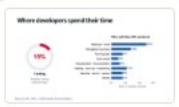
课件 p.60–61

读写代码和测试，只占工作日的 15%

5,928 个微软员工自报的工作日



2019 年的调查，现在可能不一样了 · 出处：Meyer et al., 2019，表 2



课件 p.62（中文重绘）

除了修复，还有哪些开发任务

任务	产物	环境 / 验证器	跨度
修复	补丁	仓库 / 回归测试	一个 issue
写库	一份实现	脚手架代码 / 包的测试	很多函数
生成测试	新的测试	有 bug 和修好的两版 / 能不能区分	一个行为
修 CI	代码或配置	工作流 / 必须通过的检查	一次构建
演进	不断累积的修改	一直存在的应用 / 里程碑 + 回归	互相依赖的任务



课件 p.63（中文重绘）

定位，可以单独训练

CodeScout：用强化学习训定位

搜索策略

输入：issue + 仓库
可以：查看、搜索

预测要改的位置

文件	`django/…/datetime.py`	F1
模块	`TruncDate`	F1
函数	`as_sql`	F1

✓ 奖励

三层各算一个 F1
和标准补丁里的位置比

标准补丁不给策略看

也是编程 agent，但更好做，小一点的模型也能做



课件 p.64

做应用：ViBench 评整体的"氛围编程"

需求

产品 / 功能描述

从零到一	做一个新应用	15 个应用
在参考版上做	扩展一个参考 MVP	45 个应用
在自己的版本上做	扩展 agent 之前做的 MVP	45 个应用

每个结果都用人写的浏览器测试计划来查



课件 p.65

写库：照行为重新实现

Commit0

库的脚手架：只有函数签名，函数体空着

```
def parse(…):
    ...  # 函数体空着
def evaluate(…):
    ...  # 函数体空着
```

+ 文档 + 测试 → agent 一起实现 → 跑包的测试

ProgramBench 更难

给一个参考可执行程序



从头重新实现 ⇄ 和参考程序对照

内部设计不同也行，只要可观察的行为一致

短任务越来越容易 → 这种很长的任务成了新的挑战



课件 p.66

维护：一次做一串互相依赖的 issue

一 定位 | 二 做应用 | 三 写库 | 四 维护 | 五 生成测试 | 六 修 CI

SWE-bench：一个 issue
→ 一个补丁 → 测一次

✓ M1 → ✓ M2 → ✓ M3

代码库：基础快照

Tag-M1 ✓ 测试
Tag-M2 ✓ 测试
Tag-M3 ✓ 测试

⚠ 会不会把代码库越改越乱、越来越难维护？


课件 p.67

生成测试：写好测试，仗就赢了一半

好测试

= 规定好了任务

agent

很擅长对着测试反复迭代

SWT-Bench 的例子

issue：`isValid` 现在允许结尾带换行，但只该接受字母和数字

+ 修之前的代码库

生成的测试	修之前	修之后	
`isValid("name\n") == False`	失败	通过	✓ 抓到了 bug
`isValid("name") == True`	通过	通过	
`isValid("name") == False`	失败	失败	



课件 p.68

修 CI：真正该改的是配置


```
运行环境：Node 16
包要求：Node >=18
npm ci: EBADENGINE
```

查这个包
支持的运行环境

```
# setup-node configuration
with:
  node-version: '20'
```

✓ 重新安装

✓ 构建

✓ 测试

偏运维的任务：还没有特别好的基准，要有真的基础设施来跑

课上参考：JetBrains 的 LCA CI Builds Repair



课件 p.69

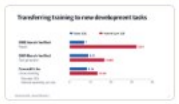
训练数据从哪来？自动造练习项目

SWE-Playground

1 项目提议	2 任务提议	3 仓库搭建	4 生成单元测试	5 实现功能	6 针对能力调整
`filesystem-simulator`	拆成 Task 1.1.1、1.1.2…	代码结构 + 函数桩	还没实现，测试先失败	读测试 → 写代码 → 验证	修 bug、写测试、从零写库…

Hybrid-Gym：把训练迁移到新任务





课件 p.72

要真跑一遍来核对

这一段记住一句

预测能省成本，
但要靠真执行兜底。



课件 p.73–79

回到开头的三个层次

写代码

代码 · 文本 · 数学混着训

分词保住空白

拉长上下文 · 学会填空

从比字面到跑测试

测试结果做强化学习

改仓库

定位 → 修改 → 验证

工具和编辑格式影响很大

SWE-bench 评测

SWE-Gym · SWE-smith 训练

多套脚手架

做开发

写代码只占 15%

前端在浏览器里验证

定位 · 应用 · 库 · 维护 · 测试 · CI

测试通过 ≠ 可以发布



课件 p.80–82

设计一个编程 agent，按六条来想

1. **代码模型**
   怎么训
2. **agent 式编程**
   用什么工具定位、修改、验证
3. **训练 agent**
   拿什么任务和奖励
4. **前端开发**
   怎么验
5. **更广的开发**
   哪些任务要覆盖
6. **预测代码行为**
   要不要拿来省成本



课件 p.80–82

CMU 11–768 · AI AGENTS

第 6 讲 · 编程 agent

下一讲：**电脑操作 agent**

每晚 20:00 更新



课件 p.80–82